

Counting Fruits with State-of-the-Art Multi-View-Stereo Approaches

Bachelor Thesis in Computer Science

presented by

Alexander Link

born on 29.08.2004 in Bamberg

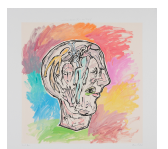
submitted to

Department of Computer Science
Professorship for Cognitive Computer Vision
Friedrich-Alexander-Universität Erlangen-Nürnberg

Academic Supervision: Prof. Dr. Bernhard Egger

Begin date: 01.05.2026

End date: 01.10.2026



Abstract

Contents

1	Introduction	1
2	Related Work	3
2.1	Fruit Counting and Yield Estimation	3
3	Fundamentals	5
3.1	VGGT	5
3.1.1	Context	5
3.1.2	Input Data	6
3.1.3	Pipeline	6
3.1.4	Output	6
3.1.5	Advantages and limitations	7
4	Method	9
4.1	Tense	9
4.2	I, We, Passive voice	9
4.3	Equations	9
4.4	Figures	9
4.5	Tables	10
5	Implementation	11
5.1	Listings	11
6	Results	13
7	Conclusions	15
7.1	Tips	15

CONTENTS

Chapter 1

Introduction

The automatic counting of fruit from images is a key task in precision agriculture, where accurate yield estimates support harvest planning and resource allocation. Reconstructing a fruit tree scene in 3D from a set of RGB images and counting the fruit it contains is appealing because it requires only inexpensive and widely available camera hardware, rather than specialized depth sensors.

citation

Recent Work such as FruitNeRF [15] show that this is possible by training a Neural Radiance Field [16] per scene and counting the fruits in the resulting volumetric representation. This approach is effective but requires a time-consuming scene-specific optimization. In this work, we instead investigate whether a feed-forward multi-view stereo model, VGGT [21], can replace scene-by-scene training. Instead of optimizing a representation of each scene, VGGT predicts a 3D reconstruction in a single forward pass. This promises a significant reduction in runtime, but raises the central question of whether the resulting geometry is sufficiently accurate for reliable counting.

Counting fruits in this scenario is challenging for several reasons: Fruits are often obscured by foliage and each other, the appearance of a scene varies with changing lighting, and the geometric complexity of a tree canopy makes a consistent reconstruction difficult. To reduce occlusion and the risk of missing apples, a scene should ideally be captured from all sides, but the number of views is limited by hardware constraints such as available memory. The cameras are therefore sparsely distributed across a full 360-degree range, and widely spaced views often capture completely different regions of the tree. This makes it difficult to establish correspondences between views

This results in two types of opposing types of errors. Individual apples observed by only a few cameras yield unstable correspondences and are easily split into multiple detections, while dense clusters of apples seen by many cameras may merge into a single detection. A single set of static parameters can therefore rarely resolve both cases optimally at the same time.

Our approach maps semantic information in the form of 2D instance masks, obtained using SAM3 [2], across views via the 3D representation predicted by VGGT. The fruits are then counted by combining a clustering step with a preceding partial instance association that supports the clustering.

Chapter 2

Related Work

2.1 Fruit Counting and Yield Estimation

Approaches to fruit counting can be distinguished based on whether they operate in the image plane or in a three-dimensional representation of the scene. Traditional image-based pipelines are divided into two subtasks: the detection of fruits in the image and their counting across multiple views [6]. While detection used to rely on handcrafted color and shape features, it is now dominated by deep neural networks [17]. For example, YOLO-based detectors achieve high accuracies on single images [1]. However, such methods count fruits on a per-frame basis and lack consistent instance tracking across multiple views, meaning that the same fruit can be counted multiple times. It is precisely this lack of cross-view tracking that is the problem addressed by the 3D-based approach pursued in this work.

This double counting is the central challenge in counting across image sequences. Liu et al. [12] introduced the first complete pipeline that tracks fruits across consecutive images and uses Structure-from-Motion (SfM) to correct double counting, more recent pipelines combine modern detectors such as YOLO with dedicated tracking modules for the same purpose [3]. In a subsequent paper, Liu et al. [13] combined detection with a semantic SfM reconstruction and assigned detections to one another using the Hungarian algorithm, using only a single camera. The association using the Hungarian algorithm is also adopted in the present work; however, there it operates on densely sampled video sequences with short frame intervals, whereas the scenario considered here is based on widely spaced 360-degree views with weak correspondences between the views.

To structurally avoid double counting, later studies shifted the counting entirely into three-dimensional space. Häni et al. [6] reconstructed rows of trees from both sides and merged the counts by combining the 3D models. Gené-Molá et al. [5] combined Mask R-CNN instance segmentation with an SfM point cloud, projecting the 2D masks into 3D space, and filtered out false detections using a support vector machine. This approach is the closest in structure to the present work, as it maps 3D instance masks to views via a 3D representation. However, it relies on an SfM reconstruction, which takes several hundred minutes per scene to complete and is therefore not suitable for a fast run, precisely the limitation that this work seeks to overcome by using a feed-forward model.

FruitNeRF [15] fits into this trend as the approach that fully integrates counting in to a neural scene representation, thereby overcoming the previous separation of reconstruction and segmentation. However, since the NeRF representation used must be optimized per scene, the computational cost

remains high. This limitaion forms the immediate starting point of this work. More recent works address this by replacing per-scene optimization with faster representations: FruitGaussian [8] and CountingFruit [11] use 3D Gaussian splatting to significantly speed up reconstruction, but they remain within the framework of scene-specific optimization.

Chapter 3

Fundamentals

3.1 VGGT

The central component of the reconstruction stage in this work is VGGT (Visual Geometry Grounded Transformer), a feed-forward transformer introduced by Wang et al. [21] and awarded the Best Paper Award at CVPR 2025. In this section, we first contextualize VGGT within the broader field of 3D reconstruction and then examine its input, data, internal pipeline, and outputs. Finally, we discuss the properties that make a suitable foundation for our coupling method.

3.1.1 Context

Reconstructing the three-dimensional structure of a scene from a set of two-dimensional images is a long-standing problem in computer vision [7]. The classical solution is a multi-stage pipeline consisting of Structure-from-Motion (SfM) and Multi-View Stereo (MVS). In the implementation by Schönberger and Frahm [20], SfM estimates the camera poses as well as a sparsely populated point cloud by extracting and matching handcrafted features across the images, followed by an iterative optimization process known as bundle adjustment. A subsequent MVS step then densifies this reconstruction. While these methods are accurate and remain widely used, they rely on iterative optimization that is computationally expensive and can take several minutes to hours per scene to complete.

More recently, learning-based approaches have replaced parts of this pipeline with neural networks. Neural Radiance Fields (NeRFs) [16] represent a scene as a volumetric radiance function encoded in the weights of a neural network, while 3D Gaussian Splatting [9] models it as a collection of colored Gaussian distributions. Both produce high-quality reconstructions but require separate optimization for each individual scene, which again incurs a substantial runtime cost per reconstruction.

A different line of work aims to eliminate per-scene optimization entirely by predicting geometry with a neural network. DUST3R [22] pioneered this *feed-forward* principle by regressing a dense pointmap from an image pair in a single network pass, followed by MAST3R [10], which added feature mapping. However, these methods operate on image pairs and still require a global alignment step to fuse the pairwise predictions into a consistent scene. VGGT [21] belongs to this feed-forward family, but generalized it from pairs to arbitrary sets of images: It processes one, a few, or up to hundreds of views together in a single forward pass and predicts a globally consistent reconstruction without any

subsequent optimization. This combination of speed and multi-view consistency motivates its use in this work, where fast reconstruction is a central goal.

3.1.2 Input Data

The input for VGGT is a sequence of N RGB images $(I_i)_{i=1}^N$ with $I_i \in \mathbb{R}^{3 \times H \times W}$, which capture the same static scene. Unlike the classical SfM pipeline, no camera parameters, calibration data, or prior pose information need to be provided: The model derives all geometric quantities directly from the image content. The number of views N is flexible and can range from a single image to several hundred, with the only practical limitation being the available GPU memory, since all views are processed jointly[21].

3.1.3 Pipeline

Internally, VGGT maps the input sequence through three conceptual stages to a set of dense 3D predictions: tokenization, alternating attention, and task-specific prediction heads. An overview is shown in Figure .

VGGT
pipeline
figure

Tokenization. Each input image I_i is first divided into non-overlapping image patches, which are encoded by a pre-trained DINOv2 backbone [18] into a set of tokens. These tokens carry local appearance and structural information for their respective image regions. For each image, the patch tokens are augmented with additional learnable tokens: a *camera token*, which aggregates the information needed to predict the camera parameters of this image, as well as a set of *register tokens*, which serve as auxiliary memory during the attention calculation and are discarded before the final prediction.

Alternating Attention. The core of the network is a transformer that processes all tokens of all images jointly. It consists of 24 attention blocks that alternate between two types of self-attention. In *frame-wise* self-attention, tokens attend only to other tokens of the same image, which preserves and refines the local structure of each image. In *global* self-attention, all tokens attend to one another across all images, thereby exchanging information between views and establishing the correspondences that link the individual images into a single, coherent scene. By alternating between these two modes, the network balances local detail with global multi-view consistency. This alternating attention design is the central architectural mechanism of VGGT; the authors show that it is superior to both purely global attention and cross-attention alternatives [21].

Prediction Heads. After the attention blocks, the refined tokens are passed to specialized prediction heads. A dedicated *camera head* processes the camera tokens to regress the intrinsic and extrinsic camera parameters of each view. A *DPT head* [19], a Dense Prediction Transformer, upsamples the image tokens back to the input resolutions to generate the dense, pixel wise outputs, namely depth maps, point maps, tracking features, and the corresponding confidence maps.

3.1.4 Output

For each input image I_i , VGGT predicts the following quantities:

- camera extrinsics and intrinsics of the view,

- a dense depth map together with a per-pixel confidence map,
- a dense point map together with a per-pixel confidence map, and
- dense feature maps used for tracking pixels across views.

The point map is the output most relevant to this work. It assigns a 3D scene coordinate $P_i(y) \in \mathbb{R}^3$ to every pixel y of an image, thereby representing a dense, pixel-wise 3D reconstruction. Crucially, the point maps of all images are expressed in a single, common coordinate system: they are defined in the reference frame of the first camera, which serves as the world origin [21]. Points observed in different views therefore already lie in a common 3D space without requiring an explicit alignment or registration step. This shared coordinate frame is the property our method builds upon, as it allows 2D instance masks from different views to be related to one another purely through their 3D coordinates.

Each dense prediction is accompanied by a confidence map that estimates the pixel-wise reliability of the reconstruction. These confidence values provide a signal for identifying and filtering out unreliable regions, such as poorly observed or geometrically ambiguous areas [21].

3.1.5 Advantages and limitations

The main advantage of VGGT for our application is its speed. Since the entire reconstruction is generated in a single forward pass, a scene can be reconstructed in a matter of seconds, whereas the SfM-based and scene-specific optimization approaches discussed in Section 3.1.1 take minutes to hours. At the same time, according to the authors, VGGT matches or exceeds the accuracy of these optimization-based alternatives across multiple 3D tasks [21], and does so without any post-processing. Another practical advantage is that it operates directly on uncalibrated images and outputs camera parameters, depth, and point maps together, so that no separate calibration or pose estimation step is required.

The fundamental limitation is that all input views are processed collectively via global self-attention, whose memory requirements increase with the number of tokens and, consequently, with the number of input views. The authors note that a naive implementation can become very memory intensive when dealing with a large number of tokens, so that in practice, the available GPU memory limits the number of images that can be reconstructed together [21]. This constraint is directly relevant to our scenario, in which a fruit tree must be captured from all sides: The limited view budget forces a sparse distribution of cameras around the scene, which in turn weakens the correspondences between neighboring views. A number of follow-up works address this scaling limit, for example by processing long sequences in segments, as in VGGT-Long [4] and VGGT-SLAM [14], or by scaling up to thousands of images as in Fast3R [24]; Pi3 [23] additionally eliminates the dependence on a fixed reference view. These extensions are beyond the scope of this work, which uses the original VGGT model.

Chapter 4

Method

4.1 Tense

Mind your use of present and past tense. Reports are typically written in present tense, except for the related work and conclusion section, which are usually in past tense. If you describe what past authors did, use past tense. If you describe what an algorithm does, which still works the same way today, use present tense.

4.2 I, We, Passive voice

It is very uncommon to write “I”. Instead, either avoid using personal pronouns by switching to a passive voice, or use “we” instead. When reading the word “we”, you can think of this as “the reader and me”.

4.3 Equations

For equations, the `align` command is recommended, since it allows you to align equations at a specific location, such as at the `=` sign.

$$y = f(x) \tag{4.1}$$

$$\boldsymbol{f} = \boldsymbol{m} \cdot \boldsymbol{a}^\top \tag{4.2}$$

Equations are referenced using `eqref`, for example Equation (4.1) and Equation (4.2). See `macros.tex` for a range of convenient macros, e.g., vector $\boldsymbol{x}$ and matrix $\boldsymbol{M}$ symbols

4.4 Figures

Figure placement is best done with the default `figure` environment. To reference a figure use `figref`, for example Figure 4.1. If possible, always create figures as vector graphics. For example an overview / pipeline graphic of your method should always be a vector graphic. Vector graphics allow zooming into figures without a loss in quality of the figure. Vector graphics can be exported in PDF format from common editing programs, which allows including them in Latex with `includegraphics`.

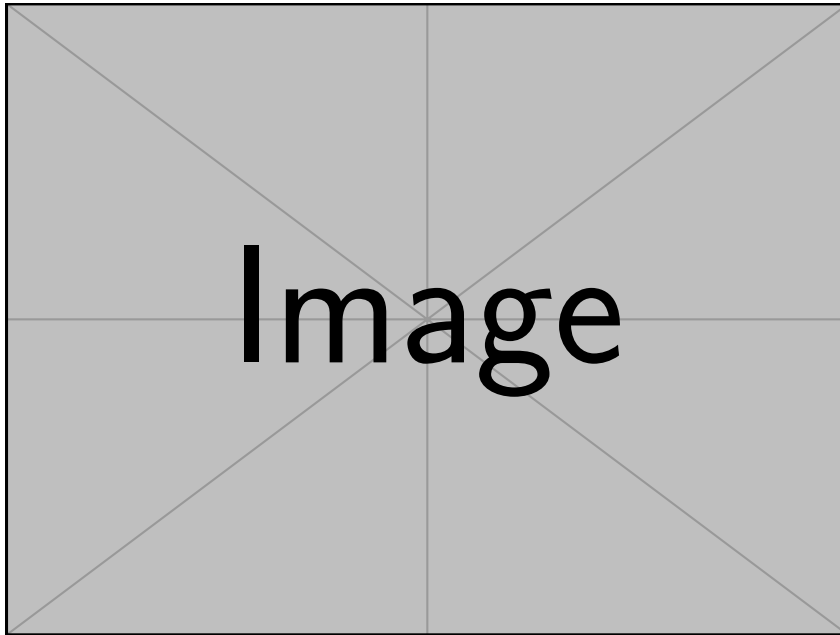


Figure 4.1: Caption of the figure. Always cite the source of the image in the caption, unless you created the image yourself.

	Metric 1↑	Metric 2↑	Metric 3↓	Metric 4↓
Method 1	23.1388	0.8611	0.2479	0.1846
Method 2	28.3813	0.9353	0.1831	0.1047
Ours	26.2468	0.9257	0.1509	0.0915

Table 4.1: Caption of the Table

Raster images should only be used in your thesis when an image is shown, that is naturally rendered at a given resolution, e.g., a result image of a rendering algorithm.

4.5 Tables

Table placement is best done with the default `table` environment. To reference a table use `tblref`, for example Table 4.1.

Chapter 5

Implementation

While the previous section aims to describe your method in a platform independent way, this section presents all the details to reproduce your work.

5.1 Listings

Not all theses require the inclusion of source code. If it is necessary, Code placement is best done with the `lstlisting` environment.

```
1 # Comment
2 def function(parameter):
3     print("This is an example Python listing")
4     return True
```

Listing 5.1: Caption of the Python Listing

```
1 // Comment
2 bool function(int parameter){
3     std::cout << "This is an example C++ listing" << std::endl;
4     return true;
5 }
```

Listing 5.2: Caption of the C++ Listing

To reference a listing use `lstref`, for example Listing 5.1.

Chapter 6

Results

Describe your qualitative and quantitative evaluations and present your results. Discuss the limitations of your work.

Chapter 7

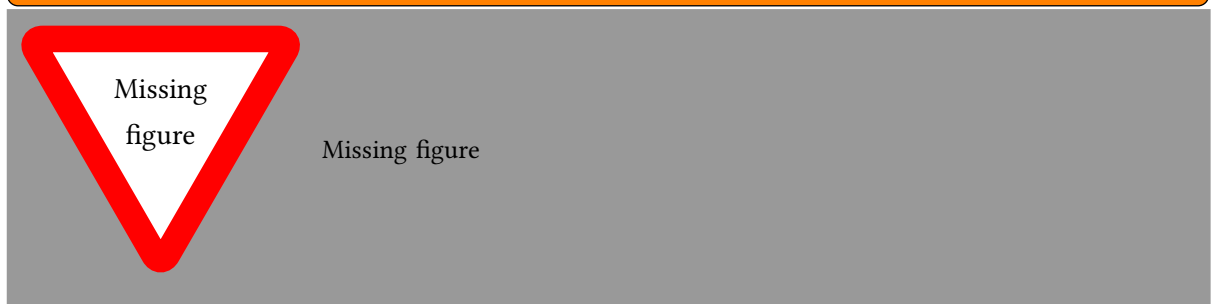
Conclusions

Give a brief summary of your work and your contributions. Afterwards, briefly outline potential opportunities for future work.

7.1 Tips

As simple package to keep track of unfinished areas of the thesis is the `todonotes` package.

Inline TODO



This is
a simple
TODO
note

Create a list of remaining TODOs with `listoftodos`. Don't forget to remove the `listoftodos` before submitting your thesis.

Todo list

citation	1
VGGT Pipeline Figure	6
This is a simple TODO note	15
Inline TODO	15
Figure: Missing figure	15

List of Figures

4.1 Caption of the Figure as listed in listoffigures 10

LIST OF FIGURES

List of Tables

4.1 Caption of the Table as listed in `listoftables` 10

LIST OF TABLES

List of Listings

5.1	Caption of the Python Listing	11
5.2	Caption of the C++ Listing	11

LIST OF LISTINGS

Bibliography

- [1] Juan Ballena-Ruiz, Juan Arcila-Diaz, and Victor Tuesta-Monteza. “Automated Detection and Counting of Gossypium barbadense Fruits in Peruvian Crops Using Convolutional Neural Networks”. In: *AgriEngineering* 7.5 (2025), p. 152. ISSN: 2624-7402. DOI: 10.3390/agriengineering7050152.
- [2] Nicolas Carion et al. *SAM 3: Segment Anything with Concepts*. 2026. arXiv: 2511.16719 [cs.CV]. URL: <https://arxiv.org/abs/2511.16719>.
- [3] G. Ramesh Chandra, K. Sri Rama Murthy, S. Archana, B. Snehitha, G. Nishanth, M. Navya, and R. Sai Charan. “Fruit Detection and Counting Using Deep Learning”. In: *2025 Eighth International Conference on Image Information Processing (ICIIP)*. 2025, pp. 175–182. DOI: 10.1109/ICIIP68302.2025.11346118.
- [4] Kai Deng, Zexin Ti, Jiawei Xu, Jian Yang, and Jin Xie. *VGGT-Long: Chunk it, Loop it, Align it – Pushing VGGT’s Limits on Kilometer-scale Long RGB Sequences*. 2025. arXiv: 2507.16443 [cs.CV]. URL: <https://arxiv.org/abs/2507.16443>.
- [5] Jordi Gené-Mola, Ricardo Sanz-Cortiella, Joan R. Rosell-Polo, Josep-Ramon Morros, Javier Ruiz-Hidalgo, Verónica Vilaplana, and Eduard Gregorio. “Fruit detection and 3D location using instance segmentation neural networks and structure-from-motion photogrammetry”. In: *Computers and Electronics in Agriculture* 169 (2020), p. 105165. ISSN: 0168-1699. DOI: 10.1016/j.compag.2019.105165.
- [6] Nicolai Häni, Pravakar Roy, and Volkan Isler. “A comparative study of fruit detection and counting methods for yield mapping in apple orchards”. In: *Journal of Field Robotics* 37.2 (Aug. 2019), pp. 263–282. ISSN: 1556-4967. DOI: 10.1002/rob.21902. URL: <http://dx.doi.org/10.1002/rob.21902>.
- [7] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. 2nd ed. Cambridge, UK: Cambridge University Press, 2004. ISBN: 978-0521540513. DOI: 10.1017/CB09780511811685.
- [8] Xiangyu Hong, Linlin Qin, Chun Shi, and Gang Wu. “FruitGaussian: 3D Gaussian Splatting for Automated Fruit Counting in Natural Orchard”. In: *Proceedings of the 44th Chinese Control Conference (CCC)*. 2025, pp. 8004–8009. DOI: 10.23919/CCC64809.2025.11179457.
- [9] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. “3D Gaussian Splatting for Real-Time Radiance Field Rendering”. In: *ACM Transactions on Graphics* 42.4 (July 2023). URL: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>.
- [10] Vincent Leroy, Yohann Cabon, and Jerome Revaud. *Grounding Image Matching in 3D with MAST3R*. 2024.

BIBLIOGRAPHY

- [11] Fengze Li, Yangle Liu, Jieming Ma, Hai-Ning Liang, Yaochun Shen, Huangxiang Li, and Zhijing Wu. *CountingFruit: Language-Guided 3D Fruit Counting with Semantic Gaussian Splatting*. 2025. arXiv: 2506.01109 [cs.CV].
- [12] Xu Liu, Steven W. Chen, Shreyas Aditya, Nivedha Sivakumar, Sandeep Dcunha, Chao Qu, Camillo J. Taylor, Jnaneshwar Das, and Vijay Kumar. *Robust Fruit Counting: Combining Deep Learning, Tracking, and Structure from Motion*. 2018. arXiv: 1804.00307 [cs.CV]. URL: <https://arxiv.org/abs/1804.00307>.
- [13] Xu Liu, Steven W. Chen, Chenhao Liu, Shreyas S. Shivakumar, Jnaneshwar Das, Camillo J. Taylor, James Underwood, and Vijay Kumar. “Monocular Camera Based Fruit Counting and Mapping With Semantic Data Association”. In: *IEEE Robotics and Automation Letters* 4.3 (2019), pp. 2296–2303. ISSN: 2377-3774. DOI: 10.1109/lra.2019.2901987. URL: <http://dx.doi.org/10.1109/LRA.2019.2901987>.
- [14] Dominic Maggio, Hyungtae Lim, and Luca Carlone. “VGGT-SLAM: Dense RGB SLAM Optimized on the SL (4) Manifold”. In: *Advances in Neural Information Processing Systems* 39 (2025).
- [15] Lukas Meyer, Andreas Gilson, Ute Schmid, and Marc Stamminger. “FruitNeRF: A Unified Neural Radiance Field based Fruit Counting Framework”. In: *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2024, pp. 1–8. DOI: 10.1109/IROS58592.2024.10802065.
- [16] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. “NeRF: representing scenes as neural radiance fields for view synthesis”. In: *Commun. ACM* 65.1 (Dec. 2021), pp. 99–106. ISSN: 0001-0782. DOI: 10.1145/3503250. URL: <https://doi.org/10.1145/3503250>.
- [17] Juan C. Miranda, Jordi Gené-Mola, Manuela Zude-Sasse, Nikos Tsoulas, Alexandre Escolà, Jaume Arnó, Joan R. Rosell-Polo, Ricardo Sanz-Cortiella, José A. Martínez-Casasnovas, and Eduard Gregorio. “Fruit sizing using AI: A review of methods and challenges”. In: *Postharvest Biology and Technology* 206 (2023), p. 112587. ISSN: 0925-5214. DOI: 10.1016/j.postharvbio.2023.112587.
- [18] Maxime Oquab et al. “DINOv2: Learning Robust Visual Features without Supervision”. In: *Transactions on Machine Learning Research* (2024). ISSN: 2835-8856.
- [19] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. “Vision Transformers for Dense Prediction”. In: *ArXiv preprint* (2021).
- [20] Johannes L. Schönberger and Jan-Michael Frahm. “Structure-from-Motion Revisited”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 4104–4113. DOI: 10.1109/CVPR.2016.445.
- [21] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. “VGGT: Visual Geometry Grounded Transformer”. In: *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025, pp. 5294–5306. DOI: 10.1109/CVPR52734.2025.00499.
- [22] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. “DUS3R: Geometric 3D Vision Made Easy”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2024, pp. 20697–20709.

- [23] Yifan Wang, Jianjun Zhou, Haoyi Zhu, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Jiangmiao Pang, Chunhua Shen, and Tong He. “ π^3 : Permutation-Equivariant Visual Geometry Learning”. In: *arXiv preprint arXiv:2507.13347* (2025).
- [24] Jianing Yang, Alexander Sax, Kevin J. Liang, Mikael Henaff, Hao Tang, Ang Cao, Joyce Chai, Franziska Meier, and Matt Feiszli. “Fast3R: Towards 3D Reconstruction of 1000+ Images in One Forward Pass”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2025.

BIBLIOGRAPHY

Declaration

I certify that I have written this thesis without outside help and without using sources other than those indicated, and that the thesis has not been submitted in the same or similar form to any other examination authority and has not been accepted by them as part of an examination. All statements that have been taken over verbatim or in spirit are marked as such.

During the preparation of this work, AI technologies were used to assist in the writing process. Specifically, <AI Tool 1 (Company, City, State, Country)> was used in order to check for grammar and style consistency, and <AI Tool 2 (Company, City, State, Country)> was used in order to assist with rephrasing and improving readability. After using these tools, the manuscript was carefully reviewed and the content was edited as needed. No tools or services were used for content generation. The content of this thesis was entirely created by myself, and I ensured that all material presented reflects my original work and research. I take full responsibility for the content of the thesis.

I agree that the thesis may be published and that it may be referred to in scientific publications.

The Friedrich-Alexander-Universität Erlangen-Nürnberg, represented by the Professorship for Cognitive Computer Vision, is granted a (non-exclusive) right to use this work and the programs created in connection with it.

Erlangen, 01.10.2026

(Alexander Link)